

Formatos de Fecha

Tipos, usos y cuándo aplicar cada uno en programación

Tipos de formato

ISO 8601 — el estándar universal

El más usado en APIs, bases de datos y comunicación entre sistemas. La Z significa UTC, +02:00 es el offset de zona horaria.

```
2026-05-27T08:30:00Z
2026-05-27T08:30:00+02:00
```

■ Siempre que envíes fechas entre frontend y backend usa este formato.

Unix Timestamp

Número de segundos (o milisegundos en JavaScript) desde el 1 de enero de 1970. Eficiente para almacenar, comparar y calcular diferencias.

```
1748339400 // segundos
1748339400000 // milisegundos (JS)
```

■ Ideal para ordenar, calcular diferencias y logs del servidor.

Formatos locales — solo para la UI

Solo para mostrar al usuario. **Nunca** para almacenar ni transmitir datos.

```
27/05/2026 // España / Europa
05/27/2026 // Estados Unidos
27 de mayo de 2026 // Formato largo ES
May 27, 2026 // Formato largo EN
```

RFC 2822 — emails y HTTP

Usado en cabeceras HTTP, emails y feeds RSS. No lo usarás directamente casi nunca pero es importante reconocerlo.

```
Tue, 27 May 2026 08:30:00 +0200
```

■ Lo verás en logs de servidor y respuestas de APIs antiguas.

Reglas de oro — cuándo usar cada uno

Cuándo	Formato recomendado
Guardar en base de datos	ISO 8601 o Unix timestamp
Enviar en una API REST	ISO 8601
Comparar o calcular diferencias	Unix timestamp
Mostrar al usuario en la UI	Formato local según idioma
Logs del servidor	ISO 8601 con timezone
Ordenar registros por fecha	Unix timestamp o timestampz

JavaScript / TypeScript

```
// ISO 8601 – para APIs y base de datos
new Date().toISOString() // "2026-05-27T08:30:00.000Z"

// Unix timestamp en milisegundos
Date.now() // 1748339400000

// Formatear para el usuario (con Intl – nativo del navegador)
new Intl.DateTimeFormat('es-ES', {
  day: 'numeric',
  month: 'long',
  year: 'numeric'
}).format(new Date()) // "27 de mayo de 2026"

// Formatear fecha y hora completa
new Intl.DateTimeFormat('es-ES', {
  dateStyle: 'full',
  timeStyle: 'short'
}).format(new Date()) // "martes, 27 de mayo de 2026, 8:30"

// Comparar dos fechas (siempre con timestamps)
const diff = Date.now() - new Date('2026-01-01').getTime()
const dias = Math.floor(diff / (1000 * 60 * 60 * 24))
```

■ Prisma / PostgreSQL

```
// Schema Prisma – siempre DateTime, nunca String
model Product {
  id String @id @default(cuid())
  createdAt DateTime @default(now()) // auto al crear
  updatedAt DateTime @updatedAt // auto al actualizar
  expiresAt DateTime? // nullable para perecederos
  publishedAt DateTime?
}

// Consultas con fechas en Prisma
// Productos que no han expirado
await prisma.product.findMany({
  where: {
    OR: [
      { expiresAt: null },
      { expiresAt: { gt: new Date() } } // mayor que ahora
    ]
  }
})

// Pedidos de los últimos 30 días
const thirtyDaysAgo = new Date(Date.now() - 30 * 24 * 60 * 60 * 1000)
await prisma.order.findMany({
  where: { createdAt: { gte: thirtyDaysAgo } }
})
```

■ PostgreSQL almacena **timestamp**tz (timestamp with time zone) cuando usas DateTime en Prisma — siempre la elección correcta para evitar problemas con cambios de horario de verano.

■■ El error más común

■ Mal

■ Bien

```
// Guardar como String
createdAt String
// "27/05/2026" → no se puede
// ordenar ni comparar
```

```
// Guardar como DateTime
createdAt DateTime
// Prisma lo convierte a
// timestampz en PostgreSQL
```

■■ Guardar fechas como strings ("27/05/2026") en la base de datos es el error más frecuente. Cuando intentas ordenar, filtrar o calcular diferencias todo se rompe. **Siempre fecha como tipo fecha, nunca como string.**